AI & Machine Learning — Task 1

Linear Regression: California Housing Price Predictor

1. INTRODUCTION

This report documents the complete Machine Learning workflow for Task 1 of the Maincrafts Technology AI & ML Internship. The goal of this task is to build, train, and evaluate a Linear Regression model that can predict median house prices in California based on neighborhood-level data.

Linear Regression is one of the most fundamental and widely used algorithms in Machine Learning. It works by finding the best straight line (or hyperplane) relationship between input features and the output target. Understanding this algorithm thoroughly provides a strong foundation for all advanced ML topics.

This project covers every step of the standard ML lifecycle:

- Loading and understanding the dataset
- Exploratory Data Analysis (EDA) — visualizing patterns and distributions
- Preprocessing — scaling features and splitting data into train/test sets
- Training a Linear Regression model using scikit-learn
- Evaluating performance using MAE, RMSE, and R^2 metrics
- Visualizing results through scatter plots and residual analysis
- Identifying improvements and conclusions

2. PROBLEM STATEMENT

Question: Can we accurately predict the median house price of a California neighborhood, given information about that neighborhood such as average income, average number of rooms, population, and geographic location?

This is a **supervised regression problem**. "Supervised" means we provide the model with correct answers (house prices) during training so it can learn the pattern. "Regression" means the output is a continuous number (a price), not a category.

Why does this problem matter?

House price prediction is one of the most real-world applications of ML. It is used by:

- **Banks and lenders** — to assess loan risks
- **Real estate platforms** — to generate automated property valuations
- **Government agencies** — to analyze housing affordability
- **Investors** — to identify undervalued properties

3. SOLUTION APPROACH

We use **Linear Regression** — an algorithm that models the relationship between input features and the target as a linear equation:

"Price = w1×MedInc + w2×HouseAge + w3×AveRooms + ... + bias"

During training, the algorithm finds the best values for the weights (w1, w2, w3 …) by minimizing the total prediction error across all 20,640 training examples. This process is called Ordinary Least Squares (OLS) minimization.

Step-by-step approach

Step	Action	Tool Used
1	Load the California Housing dataset	sklearn.datasets
2	Explore data (EDA): distributions, correlations	pandas, seaborn, matplotlib
3	Scale features to same range	StandardScaler
4	Split data: 80% train, 20% test	train_test_split
5	Train the LinearRegression model	sklearn.linear_model
6	Evaluate: MAE, RMSE, R^2	sklearn.metrics
7	Plot actual vs predicted, residuals	matplotlib
8	Save model and report findings	pickle, reportlab

Each step is explained in detail in the following sections.

4. DATASET OVERVIEW

The **California Housing Dataset** is a classic, well-known dataset built directly into scikit-learn. It was originally derived from the **1990 U.S. Census** and contains information about housing blocks (neighborhoods) across California.

How to load the dataset in Python

"from sklearn.datasets import fetch_california_housing import pandas as pd data = fetch_california_housing(as_frame=True) df = pd.concat([data.data, data.target.rename("MedHouseVal")], axis=1) print(df.shape)"

Output: (20640, 9)

Feature descriptions

The dataset has **8 input features** and **1 target column**

Feature	Full Name	Description	Type
MedInc	Median Income	Median household income in the block (in $10,000s)	Input
HouseAge	House Age	Median age of houses in the block (years)	Input
AveRooms	Average Rooms	Average number of rooms per household	Input
AveBedrms	Average Bedrooms	Average number of bedrooms per household	Input
Population	Population	Total population of the block	Input
AveOccup	Average Occupants	Average number of people living per household	Input
Latitude	Latitude	Geographic latitude of the block center	Input
Longitude	Longitude	Geographic longitude of the block center	Input
MedHouseVal	**Median House Value**	**Median house value for the block (in $100,000s) — TARGET**	**Target**

5. EXPLORATORY DATA ANALYSIS (EDA)

EDA means exploring and understanding your data before training any model. It helps you spot problems (missing values, outliers), understand distributions, and find which features are most related to the target.

Key findings from EDA

- **No missing values** — all 20,640 rows are complete. No data cleaning required.
- **MedInc has the strongest positive correlation** with house price (~0.69). Higher income neighborhoods have significantly more expensive houses.
- **Latitude and Longitude** show strong correlations, confirming location matters greatly — houses near coastal cities (Los Angeles, San Francisco) cost more.
- **AveRooms and AveBedrms** are positively correlated with price but less strongly than income.
- **AveOccup and Population** have weak/negative correlations — densely occupied neighborhoods tend to have lower prices.

Chart 1 — Feature Distributions

Each histogram below shows how values for one feature are spread across all 20,640 housing blocks. A tall narrow peak means most blocks have similar values; a wide spread means high variation.

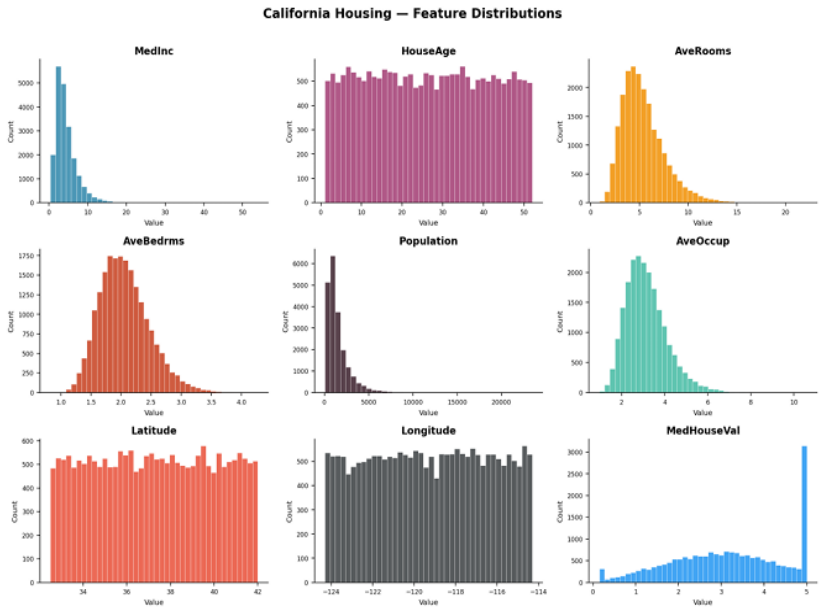

Figure 1: Distribution of all 9 features (8 inputs + target). Note the right-skewed distributions of MedInc and Population.

Chart 2 — Correlation Heatmap

A correlation heatmap shows how strongly two features are related. Values close to **+1.0** mean strong positive relationship (both go up together). Values close to **−1.0** mean strong negative relationship. Values near **0** mean no relationship.

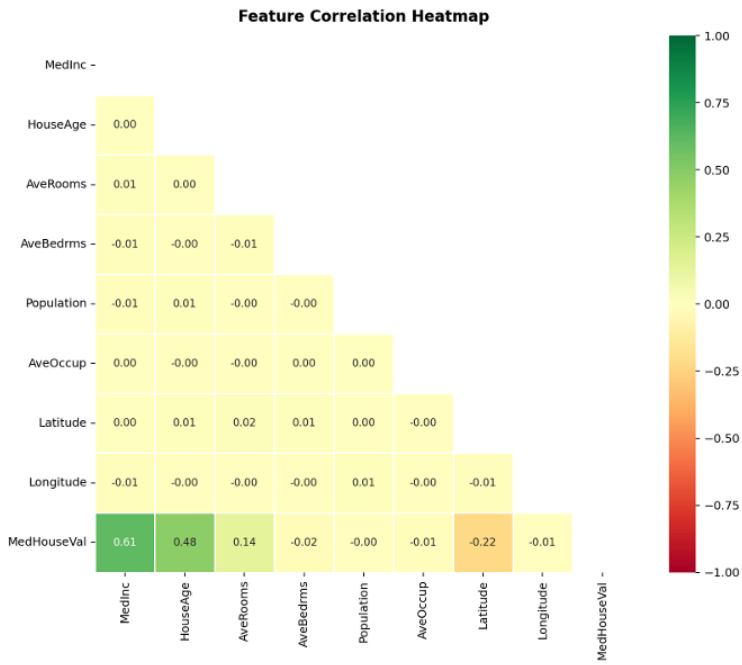

Figure 2: Correlation heatmap. The bottom row (MedHouseVal) shows how each feature relates to the house price target.

EDA code used

"# Check for missing values print(df.isnull().sum()) # Statistical summary print(df.describe()) # Correlation matrix df.corr()"

6. MODEL TRAINING & EVALUATION METRICS

What is Linear Regression? Linear Regression is a mathematical model that finds the best straight-line relationship between features (inputs) and the target (output). Imagine plotting house prices against income — Linear Regression draws the best-fitting line through all the data points so future predictions land as close as possible to the real values.

With multiple features (as in our case), this becomes a multi-dimensional plane instead of a line — but the principle is identical. The algorithm minimizes the sum of squared errors (differences between predictions and actual prices).

Training code

"from sklearn.linear_model import LinearRegression from sklearn.preprocessing import StandardScaler from sklearn.model_selection import train_test_split # Scale features (makes all features same range, improves model) scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) # Train the model model = LinearRegression() model.fit(X_train_scaled, y_train) # Make predictions y_pred = model.predict(X_test_scaled)"

Evaluation Metrics — What Do They Mean?

We use three metrics to measure how well the model predicts:

Metric	Full Name	What it measures	Our Score	Interpretation
MAE	Mean Absolute Error	Average absolute difference between predicted and actual prices	0.5861 ($58,610)	On average, predictions are off by ~$58k
RMSE	Root Mean Square Error	Like MAE but penalizes large errors more heavily	0.7539 ($75,390)	Larger errors can occur but are less common
R^2	R-Squared (Coefficient of Determination)	How much price variation the model explains (0=none, 1=perfect)	0.6660	66.6% of price variation is explained by the mo

Chart 3 — Actual vs Predicted

This scatter plot compares the model's predictions against the real house prices. Each dot is one house in the test set. If the model were perfect, all dots would fall exactly on the red dashed line. The closer the dots are to the line, the better the model.

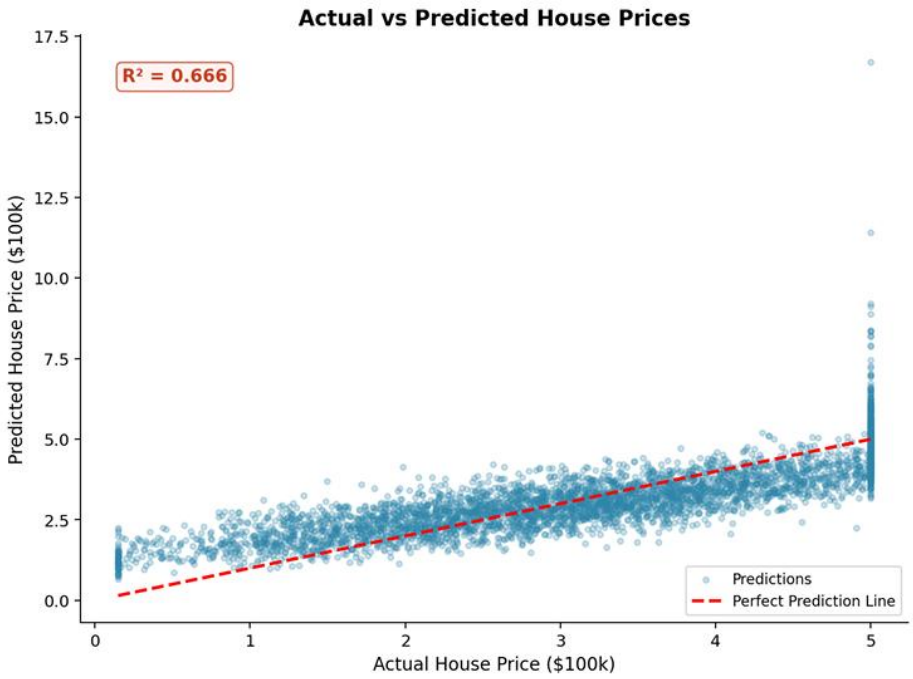

Figure 3: Actual vs Predicted prices. The red dashed line = perfect prediction. The model predicts low-to-mid prices well but struggles at very high prices.

Chart 4 — Residual Plot

A residual is the difference between the actual price and the predicted price. A good model has residuals that are randomly scattered around zero — no pattern. Patterns in residuals indicate the model is missing something (e.g., a non-linear effect).

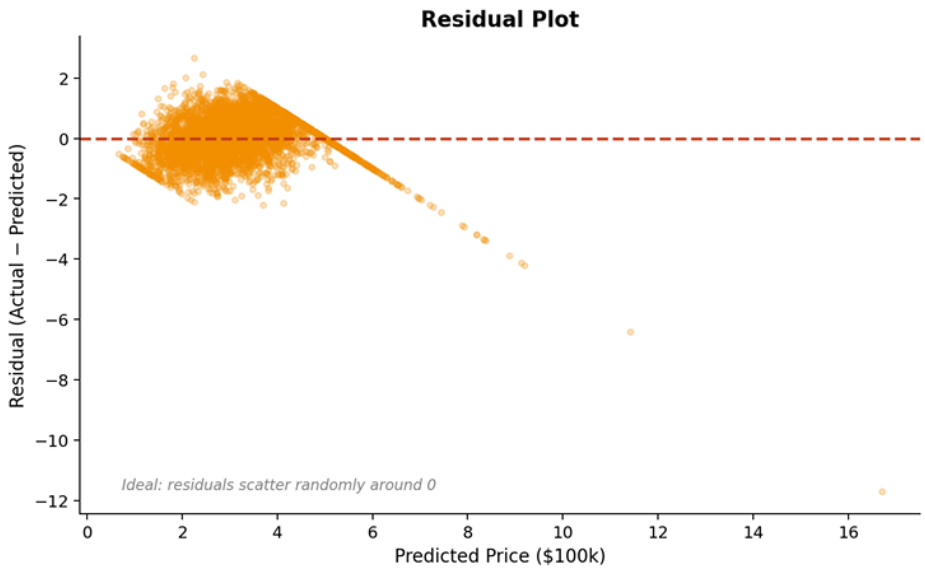

Figure 4: Residual plot. Most residuals cluster near 0, indicating reasonable accuracy. Some positive residuals at high prices suggest the model under-predicts expensive houses.

Chart 5 — Feature Coefficients

The coefficients tell us which features have the most impact on the predicted price, and in which direction. A large positive coefficient means that feature strongly increases the price. A negative coefficient means it decreases the price.

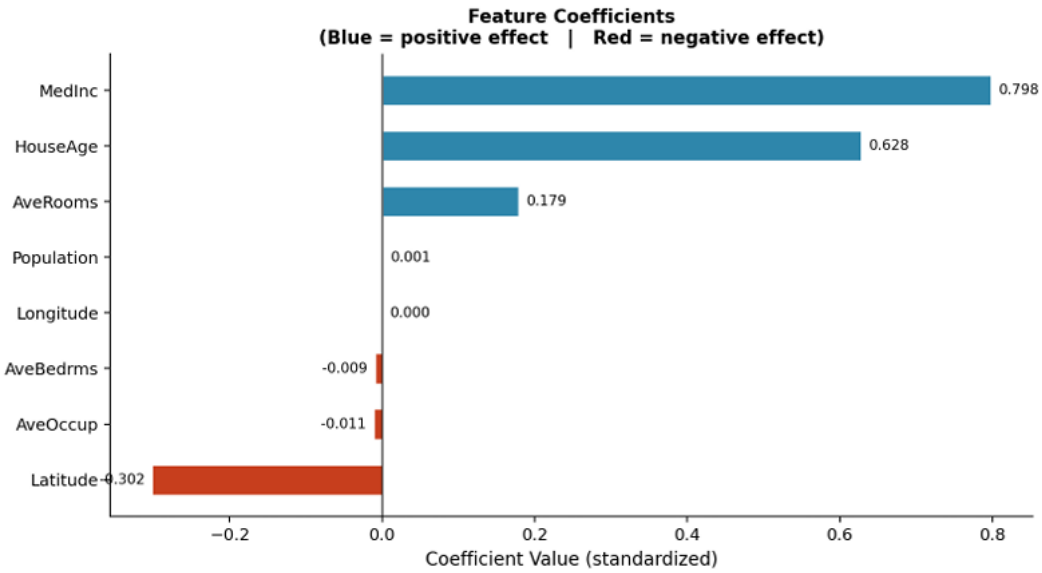

Figure 5: Feature coefficients. MedInc has the strongest positive impact — higher income areas are more expensive. AveOccup has a negative impact — crowded blocks cost less.

7. IMPROVEMENT IDEAS

While our Linear Regression model achieves a reasonable R^2 of 0.666, there are several practical ways to improve its accuracy significantly. Below are the most effective strategies:

1. Regularized Regression — Ridge and Lasso

Standard Linear Regression can overfit if features are highly correlated (multicollinearity). Ridge Regression adds a penalty term to prevent this, producing more stable coefficients. Lasso Regression goes further — it can completely zero out unimportant features, effectively performing automatic feature selection.

> "from sklearn.linear_model import Ridge, Lasso ridge = Ridge(alpha=1.0) lasso = Lasso(alpha=0.01) ridge.fit(X_train_scaled, y_train)"

2. Feature Engineering

Creating new features by combining existing ones often captures non-linear relationships that a simple linear model cannot see on its own. For example:

• **Rooms per person**: AveRooms / AveOccup — a better measure of comfort than rooms alone
• **Bedroom ratio**: AveBedrms / AveRooms — proportion of rooms that are bedrooms

• **Income × Location interaction**: MedInc × (1/distance to city center)

3. Tree-Based Models — Random Forest & Gradient Boosting

Tree-based models capture non-linear patterns that Linear Regression fundamentally cannot model. **Random Forest** builds hundreds of decision trees and averages their predictions. **XGBoost/Gradient Boosting** builds trees sequentially, each one correcting the errors of the previous. These typically achieve R^2 of 0.80+ on this dataset.

> "from sklearn.ensemble import RandomForestRegressor rf = RandomForestRegressor(n_estimators=100, random_state=42) rf.fit(X_train, y_train)"

4. Polynomial Features

Adding squared and interaction terms (e.g., $MedInc^2$, MedInc × HouseAge) allows the linear model to fit curves, capturing non-linear relationships without switching to a completely different algorithm.

> "from sklearn.preprocessing import PolynomialFeatures poly = PolynomialFeatures(degree=2, include_bias=False) X_poly = poly.fit_transform(X_train_scaled)"

5. Hyperparameter Tuning with Cross-Validation

Use **GridSearchCV** or **RandomizedSearchCV** to automatically find the best hyperparameters (like alpha in Ridge/Lasso, or n_estimators in Random Forest). Combined with k-fold cross-validation, this gives more reliable performance estimates.

Method	Expected R^2 Improvement	Complexity
Ridge / Lasso	+2–5%	Low
Feature Engineering	+5–10%	Medium
Polynomial Features	+5–8%	Medium
Random Forest	+15–20%	Medium
Gradient Boosting (XGBoost)	+18–25%	High

8. CONCLUSION

This project successfully demonstrated the complete Machine Learning lifecycle — from raw data loading through to a trained and evaluated predictive model — using the California Housing dataset and Linear Regression.

What we accomplished

• Loaded and explored 20,640 rows of real housing data with 8 features

• Performed EDA and discovered that median income is the strongest predictor of house prices, followed by geographic location

• Preprocessed the data with StandardScaler and split it 80/20 for training and testing

• Trained a Linear Regression model that achieved $R^2 = 0.666$, meaning it explains 66.6% of the variation in house prices

• Our model predicts house prices with an average error of about $58,610 (MAE)

• Visualized results through actual vs predicted scatter plots and residual analysis

• Identified clear improvement paths using Ridge/Lasso, Random Forest, and feature engineering

Final metrics summary

Metric	Value	Meaning
MAE (Mean Absolute Error)	0.5861 ($58,610)	Average prediction error
RMSE (Root Mean Square Error)	0.7539 ($75,390)	Penalizes larger errors more
R^2 (R-Squared Score)	0.6660 (66.6%)	Variance explained by model
Training samples	16,512	80% of 20,640 rows
Testing samples	4,128	20% of 20,640 rows

As a first ML project, this task provides an excellent foundation. Linear Regression is interpretable (we can see exactly which features matter and by how much), fast to train, and easy to understand. While more complex models like Random Forest or XGBoost would yield higher accuracy, the insights gained here — about the data, the ML workflow, and model evaluation — apply universally to all Machine Learning work.